

浙江大学



计算机组成与设计实验报告

项目名称: Project1 浮点加操作指令及硬件扩展设计

姓 名: 李维扬

学 号: 3230101883

学 院: 信息与工程学院

专 业: 电子科学与技术

指导老师: 赵武锋

2026 年 1 月 6 日

目 录

一	环境搭建	2
1.1	安装 wsl 系统	2
1.2	安装额外工具	2
二	译码级与信号修改	2
三	除法器修改与浮点加法器集成	3
3.1	原始顶层结构	3
3.2	新增顶层信号	3
3.3	新增输出逻辑	4
3.4	fp_adder	5
3.5	状态机修改	6
3.6	例化 fp_adder 模块	7
3.7	指令类型锁存模块 fp_operation_latch	7
3.8	小结	8
3.9	实验结果	9
四	扩展：集成浮点乘法器和除法器	9
4.1	新增输出与完成信号	10
4.2	为乘法/除法增加独立的 stb 延迟链	10
4.3	复用操作数锁存逻辑	11
五	新增测试文件	13
5.1	测试文件编写	13
5.2	实验结果	15
六	问题反馈与解决	17
6.1	Linux 版本与兼容性问题	17
6.2	新增.S 测试文件问题	18

一 环境搭建

1.1 安装 wsl 系统

wsl 是 Windows Subsystem for Linux 2 的简称, 是 Windows10/11 支持的轻量级 Linux 虚拟机, 非常适合中小型体量的 Linux 工程搭建和运行。

以管理员身份打开 PowerShell, 输入 `wsl --install Ubuntu 20.04`。linux 系统的兼容性问题比较令人头疼, 所以要注意安装的版本必须为 20.04, 否则后续的软件无法安装运行。

参考教程, 下载完成后, Windows 终端输入 `wsl` 或 `bash` 即可重新进入 Ubuntu, 后续的终端操作都在 Ubuntu 中进行。

运行 `sudo apt update`, 更新 Ubuntu 的下载库; 运行 `sudo apt install -y build-essential`, 下载基本的系统工具。

1.2 安装额外工具

在 vscode 中搜索下载 WSL 扩展和 Remote Explorer 扩展, 完成后点击左侧小电视即可进入 Ubuntu 系统。将 project1 随行的文件夹 RVfpga_NexysA7-DDR 在 Ubuntu 系统中打开, 挂载工程目录。

然后在 linux 系统中通过命令安装 Catapult-studio, Verilator, Gtkwave 等工具并运行, 环境配置完成。

二 译码级与信号修改

本次 project 基于著名的开源 Soc 项目 VeeRwolf, VeeRwolf 所采用的 RISC-V 核心是 EL2。EL2 是一个四级流水线的简单但完整的 RISC-V 处理器, 支持 RV32IMC 指令集, 但是不原生支持浮点操作。本项目中我们按照指示, 完成 EL2 的指令集及硬件集成, 使之支持单精度的基于 IEEE 754 标准的浮点加操作。

我们重点关注 `src/VeeRwolf/VeeR_EL2CoreComplex/exu/el2_exu_div_ctl.sv` 这个文件。它是 EL2 的整数除法器, 一个非流水线操作的模块, 我们将在这个模块中实现浮点加模块的例化。浮点硬件将利用整数除法器的信号完成集成。

首先我们参照项目说明的指示添加 `fp_add`, `fp_mul`, `fp_div` 指令 (即浮点加、浮点乘、浮点除) 的外部接口, 再定义它们的逻辑。这一步骤严格按《项目说明》中执行即可。

三 除法器修改与浮点加法器集成

在接口连接好之后，我们要正式开始将浮点加法器集成到除法器中。

3.1 原始顶层结构

原目标除法器文件 `el2_exu_div_ctl` 只负责整数除法运算：

- 输入：dividend（被除数）、divisor（除数）、以及来自执行单元的控制包 `dp`；
- 输出：
 - `finish_dly`：整数除法完成信号；
 - `out_raw[31:0]`：整数除法结果；
 - `finish_dly_div_fp` 和 `out[31:0]` 直接转发这两个信号。

现在希望在这个模块中复用同一套输入接口，同时要求

- 整数除法和浮点加法都是非流水线模块，即一次只处理一条指令；
- 使用一个统一的“完成信号” `finish_dly_div_fp` 来表示：
 - 当进行整数除法时：`finish_dly = 1` 表示完成；
 - 当进行浮点加法时：`fp_adder` 的 `output_z_stb = 1` 表示完成；
- 输出结果端口 `out[31:0]` 也要根据当前操作类型自动选择整数除法结果或浮点加法结果。

原始文件中顶层控制大致为：

```
1 logic [31:0] out_raw;
2 logic      finish_dly;
3
4 assign finish_dly_div_fp = finish_dly;
5 assign out[31:0]        = {32{finish_dly}} & out_raw[31:0];
```

即整数除法完成时，`finish_dly` 拉高 1 个周期。`finish_dly` 直接传给 `finish_dly_div_fp`，而结果 `out` 只有在 `finish_dly` 为 1 时才有效（通过 `{32{finish_dly}} & out_raw` 实现按位屏蔽）。

我们加上浮点加法后还要增加区分功能，区分当前是整数除法还是浮点操作。

3.2 新增顶层信号

在顶层定义中，我们新增如下信号：

```
1 logic [31:0] out_fpadd;      // 浮点加法的输出结果
2 logic      fpadd_done;      // 浮点加法完成标志 (fp_adder 的 output_z_stb)
3
4 logic      stb;              // 来自 fp 控制包的“浮点加法开始”脉冲
```

```

5 logic      stb_d;           // stb 延迟 1 拍
6 logic      stb_dd;          // stb 延迟 2 拍
7 logic      stb_for_fp;      // 最终送给 fp_adder 的 input*_stb
8
9 logic [31:0] divisor_d;      // divisor 延迟 1 拍
10 logic [31:0] divisor_dd;     // divisor 延迟 2 拍
11 logic [31:0] dividend_d;     // dividend 延迟 1 拍
12 logic [31:0] dividend_for_fp; // 供 fp_adder 使用的被加数
13 logic [31:0] divisor_for_fp; // 供 fp_adder 使用的加数
14
15 logic [3:0] active_operation; // 当前指令类型编码

```

其中包含

- (1) 浮点加法结果与完成标志: out_fpadd、fpadd_done;
- (2) 拉长控制脉冲、锁存操作数: stb 相关和 dividend/divisor 延迟相关;
- (3) 指令类型编码: active_operation, 用于选择当前是整数除法还是浮点加法(可扩展乘法除法)。

3.3 新增输出逻辑

根据上述输入, 将输出逻辑改为:

```

1 assign finish_dly_div_fp = (active_operation[3] & finish_dly) |
2                             (active_operation[2] & fpadd_done);
3
4 assign out[31:0] = ({32{active_operation[3] & finish_dly}} &
5 out_raw[31:0]) |
6                     ({32{active_operation[2] & fpadd_done}} &
7 out_fpadd[31:0]);

```

3.3.1 active_operation

active_operation 通过文件末尾定义的模块 fp_operation_latch 赋值:

```

1 case ({fp_add, fp_mul, fp_div})
2   3'b100: a <= 4'b0100; // 浮点加法
3   3'b010: a <= 4'b0010; // 浮点乘法
4   3'b001: a <= 4'b0001; // 浮点除法
5   ...
6 endcase
7
8 // 如果 fp_add/fp_mul/fp_div 全为 0, 则:
9 // a <= 4'b1000; // 即 active_operation[3] = 1

```

其中

- active_operation[3] = 1: 当前指令是整数除;
- active_operation[2] = 1: 当前指令是浮点加, 走 fp_adder;

- 其他位预留给浮点乘法、浮点除法。

3.3.2 完成信号

finish_dly_div_fp 作为统一的完成信号。

- 当 active_operation[3] = 1, 并且除法模块 finish_dly = 1 时:
 - (active_operation[3] & finish_dly) = 1, 完成信号有效;
- 当 active_operation[2] = 1, 并且 fpadd_done = 1 时:
 - (active_operation[2] & fpadd_done) = 1, 完成信号有效;
- 两者进行或操作, 得到统一的完成输出。

在原整数除法指令不变 (只增加一层选择) 的同时, 实现了浮点加法。完成信号由 fp_adder 的 output_z_stb 驱动。

3.3.3 结果输出选择

out[31:0] 通过按位屏蔽选择对应数据。

- 当 active_operation[3] & finish_dly = 1 时:
 - {32{active_operation[3] & finish_dly}} = 全 1, out_raw 透传;
- 当 active_operation[2] & fpadd_done = 1 时:
 - {32{active_operation[2] & fpadd_done}} = 全 1, out_fpadd 透传;

与原来完全一致, 只是多了一层操作类型选择。

3.4 fp_adder

fp_adder 是一个状态机模块。

```

1 input  [31:0] input_a, input_b;    // 两个操作数
2 input          input_a_stb;        // A 有效时拉高
3 input          input_b_stb;        // B 有效时拉高
4 output         input_a_ack;        // A 已被接收
5 output         input_b_ack;        // B 已被接收
6
7 output [31:0] output_z;            // 输出结果
8 output         output_z_stb;       // 结果有效
9 input          output_z_ack;       // 下游接受结果
10
11 input         clk, rst;
```

其状态有:

- get_a: 等待 input_a_stb && input_a_ack, 然后锁存 input_a;
- get_b: 等待 input_b_stb && input_b_ack, 然后锁存 input_b;
- 然后进入 unpack、add、round、pack 等状态
- 最后在 put_z 状态下拉高 output_z_stb, 等待 output_z_ack (在这里固定为 1)。

可以看到，fp_adder 顺序地读取 input_a 和 input_b，它需要 input_a_stb 和 input_b_stb 在一段时间内保持为 1 以完成握手、锁存操作。而 e12_exu_div_ctl 中的原始操作数 dividend、divisor 以及 fp.fp_add 控制信号很快就会消失。

如果直接把 input_a = dividend, input_b = divisor, input*_stb = fp.fp_add 连接，那么 fp_adder 很可能还没来得及在 get_a 状态下握手，控制信号就已经无效了。这样将导致读取失败或读取到错误数据。

3.5 状态机修改

为了解决上面的问题，我们需要做一些额外处理。

3.5.1 拉长 fp_add 脉冲

注意到有如下加法开始信号。

```

1  assign stb = fp.fp_add;
2
3  always @(posedge clk) begin
4      stb_d      <= stb;
5      stb_dd     <= stb_d;
6      divisor_d  <= divisor;
7      divisor_dd <= divisor_d;
8      dividend_d <= dividend;
9  end
10
11 assign stb_for_fp = stb | stb_d | stb_dd;
```

其中 stb 是原始的一拍控制信号，指示有一条 fp_add 指令，可以用两个触发器 stb_d、stb_dd 把它延长为 3 个周期。最终的 stb_for_fp = stb OR stb_d OR stb_dd。当有浮点加法指令时，stb_for_fp 至少保持 2-3 个周期为 1，fp_adder 在 get_a 或 get_b 状态就有足够时间完成握手。

3.5.2 锁存和保持操作数

除数和被除数也需要保持稳定。处理方法类似：

```

1  assign divisor_for_fp = divisor | divisor_d | divisor_dd;
2  assign dividend_for_fp = dividend | dividend_d;
```

- divisor (加数 B): 使用两级寄存 divisor_d、divisor_dd 并或运算，保证在 stb_for_fp 有效期间，数值稳定；
- dividend (加数 A): 使用一级寄存 dividend_d 并或运算，同样在 1-2 个周期内保持稳定；

最终传递给 fp_adder 的不是瞬态的 dividend 和 divisor，而是在时间上被拉长的 dividend_for_fp 和 divisor_for_fp。

这样，即使上游只在一个时钟周期内给出操作数，也能在接下来的几个周期内完成状态机的 get_a、get_b，保证操作数不会丢失或出现错读。

3.6 例化 fp_adder 模块

在上面的基础上，可以开始例化 fp_adder：

```
1 fp_adder simple_fp_adder (
2     .input_a      ( dividend_for_fp ),
3     .input_b      ( divisor_for_fp ),
4     .input_a_stb   ( stb_for_fp      ),
5     .input_b_stb   ( stb_for_fp      ),
6     .output_z_ack  ( 1'b1            ),
7     .clk           ( clk              ),
8     .rst           ( ~rst_l           ),
9     .output_z      ( out_fpadd        ),
10    .output_z_stb   ( fpadd_done       ),
11    .input_a_ack    (                  ),
12    .input_b_ack    (                  )
13 );
```

- (1) 输入操作数：A 对应 dividend_for_fp，B 对应 divisor_for_fp。
- (2) 输入握手信号：input_a_stb 和 input_b_stb 都接 stb_for_fp，同一条指令的两个操作数同时有效。
- (3) 输出握手信号：output_z_ack 固定接高电平 1'b1，表示随时准备好接收结果。这样 fp_adder 在完成计算后，output_z_stb 拉高一个周期即可，然后状态机回到 get_a。
- (4) 复位信号：原有 EXU 采用 rst_l 为低有效复位，因此这里对 fp_adder 的 rst 取反：~rst_l。
- (5) 结果与完成标志：
 - 结果输出 out_fpadd；
 - 完成标志 fpadd_done，直接接到前面统一完成逻辑中。

3.7 指令类型锁存模块 fp_operation_latch

为了在同一个 el2_exu_div_ctl 里同时支持整数除法和浮点运算，我们需要一个简单的操作类型状态机，根据 fp 控制包中的位决定当前操作类型。

在文件末尾添加了模块：

```
1 module fp_operation_latch (
2     input wire clk,           // 时钟
3     input wire dp_valid,      // 数据有效信号
4     input wire fp_add,        // 浮点加法操作
5     input wire fp_mul,        // 浮点乘法操作
```



```

6     input wire fp_div,          // 浮点除法操作
7     output reg [3:0] a          // 4位输出信号
8 );
9
10    wire add_only, mul_only, div_only;
11    wire unique_operation;
12    wire all_zero;
13
14    assign add_only = fp_add & ~fp_mul & ~fp_div;
15    assign mul_only = ~fp_add & fp_mul & ~fp_div;
16    assign div_only = ~fp_add & ~fp_mul & fp_div;
17    assign unique_operation = add_only | mul_only | div_only;
18
19    assign all_zero = ~fp_add & ~fp_mul & ~fp_div;
20
21    always @(posedge clk) begin
22        if (dp_valid) begin
23            if (unique_operation) begin
24                case ({fp_add, fp_mul, fp_div})
25                    3'b100: a <= 4'b0100;
26                    3'b010: a <= 4'b0010;
27                    3'b001: a <= 4'b0001;
28                    default: a <= a;
29                endcase
30            end else if (all_zero) begin
31                a <= 4'b1000; // 非浮点运算，走整数除法路径
32            end else begin
33                a <= a;        // 多个浮点操作同时为 1，保持上一状态
34            end
35        end else if (all_zero) begin
36            a <= a;          // 空闲周期保持不变
37        end
38    end
39
40 endmodule

```

- 每当 `dp_valid = 1` 时：
 - 如果只选中了一个浮点操作 (`fp_add` , `fp_mul` , `fp_div` 之一): 根据是哪一个, 将 `a` 设置成不同的 4 位编码。
 - 如果三个都为 0: 设置 `a = 4'b1000`, 表示当前是整数除法。
 - 如果多个浮点位同时为 1, 则认为输入非法, 保持上一状态。
- `a[3]`用来标识当前是整数除法路径, 用于选择 `finish_dly / out_raw`;
- `a[2]`用来标识当前是浮点加法路径, 用于选择 `fpadd_done / out_fpadd`;
- `a[1]`、`a[0]`保留给浮点乘法除法的扩展使用。

3.8 小结

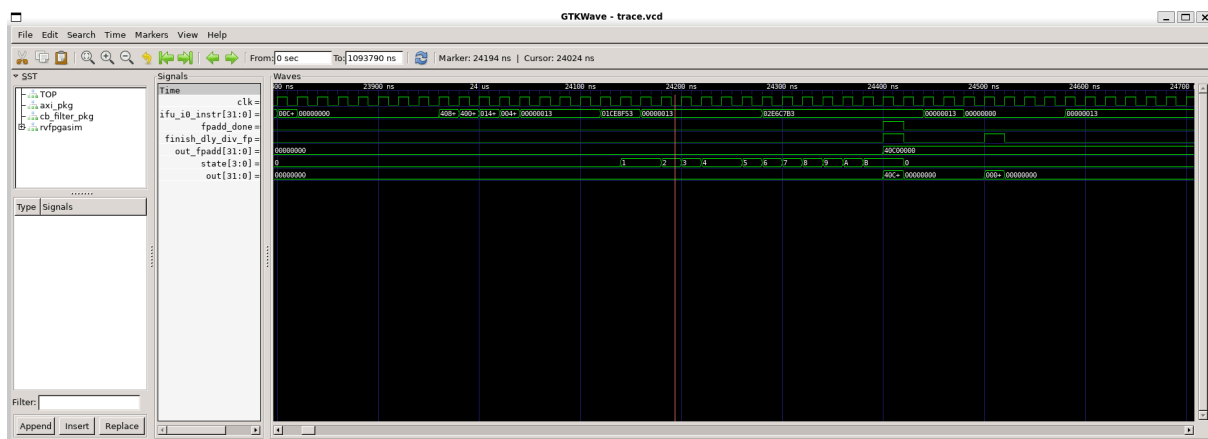
修改后的 `e12_exu_div_ctl` 总体结构如下。

- (1) 输入：上游通过 `dp.valid` 和 `fp` 中的标志位发出一条指令，可能是整数除法也可能是浮点加法。
- (2) 类型判定：`fp_operation_latch` 根据 `fp_add/fp_mul/fp_div`，把当前操作类型编码到 `active_operation` 里。
- (3) 运算
 - 整数除法：通过原有模块，最终产生 `finish_dly` 和 `out_raw`；
 - 浮点加法：
 - `fp.fp_add` 产生一个 `stb` 脉冲；
 - `stb`、`dividend`、`divisor` 时序延长、锁存；
 - `fp_adder` 在几个周期内顺利读入，完成浮点加并输出 `fpadd_done` 和 `out_fpadd`。
- (4) 结果选择：根据 `active_operation`，统一生成完成信号 `finish_dly_div_fp`，统一选择输出结果 `out[31:0]`。

这样就在不改变原有整数除法的前提下集成了单精度浮点加法器。

3.9 实验结果

按照实验说明通过终端控制 Verilator 编译并运行，打开 gtkwave 窗口得到波形图：



与实验说明中的示例一致，说明浮点加法器集成没有问题。

四 扩展：集成浮点乘法器和除法器

在完成浮点加法器 `fp_adder` 的集成之后，我们采用同样的思路，将浮点乘法器 `multiplier` 和浮点除法器 `divider` 集成进 `e12_exu_div_ctl`，为 `fp_mul` 和 `fp_div` 指令提供硬件支持。

4.1 新增输出与完成信号

在顶层信号定义中，为乘法和除法各增加了一组结果 + 完成标志信号：

```
1 logic [31:0] out_fpmul;
2 logic      fpmul_done;
3 logic [31:0] out_fpdiv;
4 logic      fpdiv_done;
```

这样，每种运算类型都有自己的完成标志

- 整数除法：finish_dly + out_raw;
- 浮点加法：fpadd_done + out_fpadd;
- 浮点乘法：fpmul_done + out_fpmul;
- 浮点除法：fpdiv_done + out_fpdiv。

随后把统一完成信号 finish_dly_div_fp 扩展为：

```
1 assign finish_dly_div_fp = (active_operation[3] & finish_dly) |
2                             (active_operation[2] & fpadd_done) |
3                             (active_operation[1] & fpmul_done) |
4                             (active_operation[0] & fpdiv_done);
```

并将统一结果输出 out[31:0] 扩展为四路选择。

```
1 assign out[31:0] = ({32{active_operation[3] & finish_dly}} &
2 out_raw[31:0]) |
3                  ({32{active_operation[2] & fpadd_done}} &
4 out_fpadd[31:0]) |
5                  ({32{active_operation[1] & fpmul_done}} &
6 out_fpmul[31:0]) |
7                  ({32{active_operation[0] & fpdiv_done}} &
8 out_fpdiv[31:0]);
```

仍然用 active_operation 的不同位指示当前是哪种运算，用每种运算自己的完成标志去打开一组 32 位的掩码，把所有结果或运算形成最终输出。

4.2 为乘法/除法增加独立的 stb 延迟链

浮点乘法器 multiplier 和浮点除法器 divider 的接口形式与 fp_adder 非常相似，使用 input_a / input_b + input_a_stb / input_b_stb 输入操作数，内部用状态机依次读取两个操作数再进行计算，output_z + output_z_stb 输出结果。

因此它们同样需要延长 stb 有效时间，参考前文对 fp.fp_add 的做法，对 fp.fp_mul 和 fp.fp_div 各增加一组延迟链即可。

```

1 logic stb_mul, stb_mul_d, stb_mul_dd, stb_mul_for_fp;
2 logic stb_div, stb_div_d, stb_div_dd, stb_div_for_fp;
3
4 assign stb      = fp.fp_add;
5 assign stb_mul  = fp.fp_mul;
6 assign stb_div  = fp.fp_div;
7
8 always @(posedge clk) begin
9     stb_d      <= stb;
10    stb_dd     <= stb_d;
11    stb_mul_d   <= stb_mul;
12    stb_mul_dd  <= stb_mul_d;
13    stb_div_d   <= stb_div;
14    stb_div_dd  <= stb_div_d;
15    // 这里顺便锁存 dividend / divisor, 见前一节
16 end
17
18 assign stb_for_fp      = stb      | stb_d      | stb_dd;
19 assign stb_mul_for_fp = stb_mul   | stb_mul_d   | stb_mul_dd;
20 assign stb_div_for_fp = stb_div   | stb_div_d   | stb_div_dd;

```

- stb_for_fp 提供给加法器 fp_adder
- stb_mul_for_fp 提供给乘法器 multiplier
- stb_div_for_fp 提供给除法器 divider

三者互不干扰。

4.3 复用操作数锁存逻辑

处理浮点加法时我们已经用 divisor_d / divisor_dd 和 dividend_d 做了操作数锁存：

```

1 always @(posedge clk) begin
2     divisor_d <= divisor;
3     divisor_dd <= divisor_d;
4     dividend_d <= dividend;
5 end
6
7 assign divisor_for_fp = divisor | divisor_d | divisor_dd;
8 assign dividend_for_fp = dividend | dividend_d;

```

加、乘、除三个浮点模块都共享 dividend_for_fp / divisor_for_fp 信号即可。

当 fp_add、fp_mul 或 fp_div 其中之一拉高时，对应的 stb*_for_fp 会在数个周期内保持 1。同一时间里，dividend_for_fp 和 divisor_for_fp 也由于锁存逻辑而保持不变，三个浮点模块的状态机都可以在这段时间内完成读取。

4.3.1 例化乘法器与除法器

参考加法器的例化，按照同样的方式例化浮点乘法器与除法器。

```

1 multiplier simple_fp_multiplier (
2     .input_a      ( dividend_for_fp ),
3     .input_b      ( divisor_for_fp ),
4     .input_a_stb   ( stb_mul_for_fp ),
5     .input_b_stb   ( stb_mul_for_fp ),
6     .output_z_ack  ( 1'b1 ),
7     .clk           ( clk ),
8     .rst           ( ~rst_l ),
9     .output_z      ( out_fpmul ),
10    .output_z_stb   ( fpmul_done ),
11    .input_a_ack    ( ),
12    .input_b_ack    ( ),
13 );
14
15 divider simple_fp_divider (
16     .input_a      ( dividend_for_fp ),
17     .input_b      ( divisor_for_fp ),
18     .input_a_stb   ( stb_div_for_fp ),
19     .input_b_stb   ( stb_div_for_fp ),
20     .output_z_ack  ( 1'b1 ),
21     .clk           ( clk ),
22     .rst           ( ~rst_l ),
23     .output_z      ( out_fpdiv ),
24     .output_z_stb   ( fpdiv_done ),
25     .input_a_ack    ( ),
26     .input_b_ack    ( ),
27 );

```

除了使用不同的 `stb*_for_fp` 和输出信号名称外，它们的结构与 `fp_adder` 非常相似。

4.3.2 与 `fp_operation_latch` 对应

前面定义 `fp_operation_latch` : `a[3]`: 整数除法, `a[2]`: 浮点加法, `a[1]`: 浮点乘法, `a[0]`: 浮点除法。

现在把四种运算的完成标志、结果输出接入:

- `finish_dly + out_raw` $\longleftrightarrow$ `active_operation[3]`;
- `fpadd_done + out_fpadd` $\longleftrightarrow$ `active_operation[2]`;
- `fpmul_done + out_fpmul` $\longleftrightarrow$ `active_operation[1]`;
- `fpdiv_done + out_fpdiv` $\longleftrightarrow$ `active_operation[0]`。

因此, 只要 `fp_operation_latch` 编码正确, `el2_exu_div_ctl` 顶层就可以自动根据指令类型选择正确的运算结果与完成信号。

五 新增测试文件

5.1 测试文件编写

为了验证我们集成的三个浮点运算模块（加法、乘法、除法）都能正常工作，需要编写一个综合测试文件 test.S。

5.1.1 测试数据与预期结果

使用两个简单的 IEEE 754 单精度浮点数作为操作数。

- 0x40800000: 符号位=0, 指数=10000001(129), 尾数=00000..., 即 $(-1)^0 \times 1.0 \times 2^{\{129-127\}} = 4.0$
- 0x40000000: 符号位=0, 指数=10000000(128), 尾数=00000..., 即 $(-1)^0 \times 1.0 \times 2^{\{128-127\}} = 2.0$

预期结果:

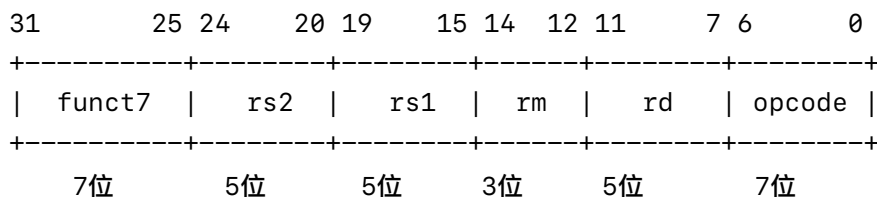
指令	运算	结果 (float)	结果 (hex)
fadd.s	$4.0 + 2.0$	6.0	0x40C00000
fmul.s	4.0×2.0	8.0	0x41000000
fdiv.s	$4.0 \div 2.0$	2.0	0x40000000
div (int)	$20 \div 4$	5	0x00000005

整数除法 div a5, a3, a4 用于验证原有的整数除法功能不受影响。

上述结果将在波形图中得到验证。

5.1.2 RISC-V 指令编码

参考课程内容，RISC-V F 扩展浮点指令的编码格式（R-type）如下：



其中 funct7 区分不同的浮点运算：

指令	funct7
fadd.s	0000000
fmul.s	0001000
fdiv.s	0001100

5.1.2.1 fadd.s 编码示例

fadd.s t5, t4, t3: 计算 $t4 + t3$, 结果存入 t5。

```
funct7 = 00000000
rs2     = 11100   (t3 = x28)
rs1     = 11101   (t4 = x29)
rm      = 000
rd      = 11110   (t5 = x30)
opcode  = 1010011
```

拼接: 00000000 | 11100 | 11101 | 000 | 11110 | 1010011

十六进制: 0x01CE8F53

5.1.2.2 fmul.s 编码示例

fmul.s s0, t4, t3: 计算 $t4 \times t3$, 结果存入 s0。

```
funct7 = 0001000
rs2     = 11100   (t3 = x28)
rs1     = 11101   (t4 = x29)
rm      = 000
rd      = 01000   (s0 = x8)
opcode  = 1010011
```

拼接: 0001000 | 11100 | 11101 | 000 | 01000 | 1010011

十六进制: 0x11CE8453

5.1.2.3 fdiv.s 编码示例

fdiv.s s1, t3, t4: 计算 $t3 \div t4$, 结果存入 s1。

```
funct7 = 0001100
rs2     = 11101   (t4 = x29)
rs1     = 11100   (t3 = x28)
rm      = 000
rd      = 01001   (s1 = x9)
opcode  = 1010011
```

拼接: 0001100 | 11101 | 11100 | 000 | 01001 | 1010011

十六进制: 0x19DE04D3

5.1.3 测试文件说明

test.S 的主体结构如下:

```
1  .globl main
2  main:
3      li t3, 0x40800000    # t3 = 4.0
4      li t4, 0x40000000    # t4 = 2.0
```

```

5      li a3, 20          # 整数除法被除数
6      li a4, 4           # 整数除法除数
7
8  REPEAT:
9      INSERT_NOPs_4      # 插入 4 个 NOP, 稳定流水线
10
11     .word 0x01CE8F53    # fadd.s t5, t4, t3
12     INSERT_NOPs_10     # 等待加法器完成
13
14     .word 0x11CE8453    # fmul.s s0, t4, t3
15     INSERT_NOPs_10     # 等待乘法器完成
16
17     .word 0x19DE04D3    # fdiv.s s1, t3, t4
18     INSERT_NOPs_10     # 等待除法器完成
19
20     div a5, a3, a4      # 整数除法
21     INSERT_NOPs_10
22
23     beq zero, zero, REPEAT # 无限循环

```

由于浮点运算是非流水线的，需要多个周期才能完成，我们插入多个 NOP 确保每条浮点指令有足够时间执行，可能会有比较长的间隔。为了便于观察结果，我们让这个过程无限循环。

5.1.4 验证方法

5.1.4.1 寄存器值

在仿真器或调试器中查看以下寄存器：

- t5 (x30)：应为 0x40C00000 (fadd.s 结果 = 6.0)
- s0 (x8)：应为 0x41000000 (fmul.s 结果 = 8.0)
- s1 (x9)：应为 0x40000000 (fdiv.s 结果 = 2.0)
- a5 (x15)：应为 0x00000005 (div 结果 = 5)

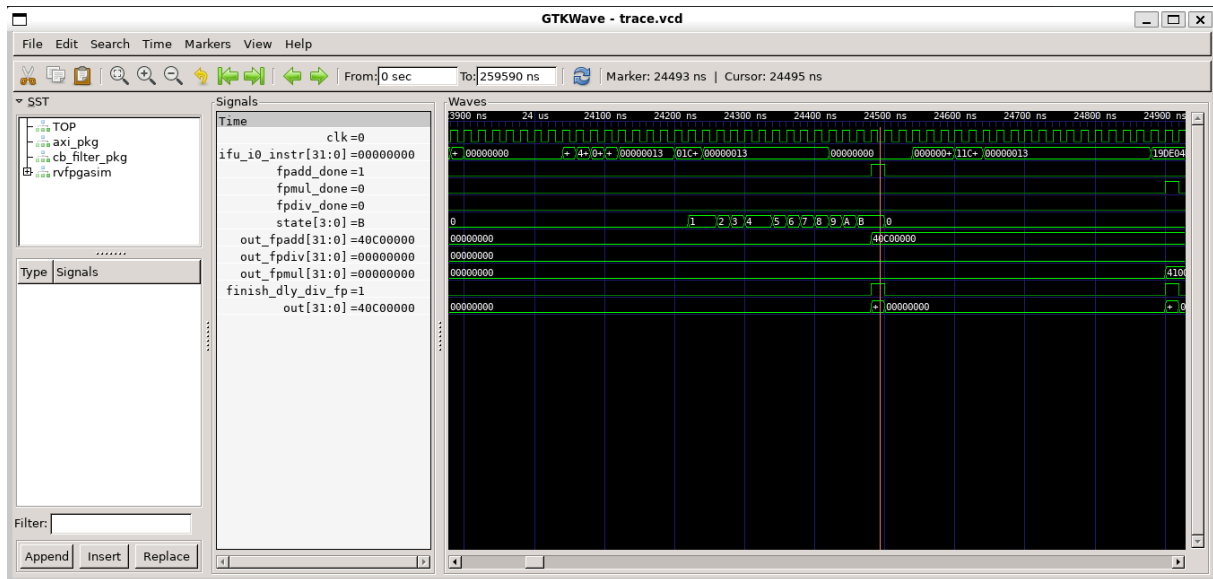
5.1.4.2 波形信号

每条浮点指令执行时应该能观察到：

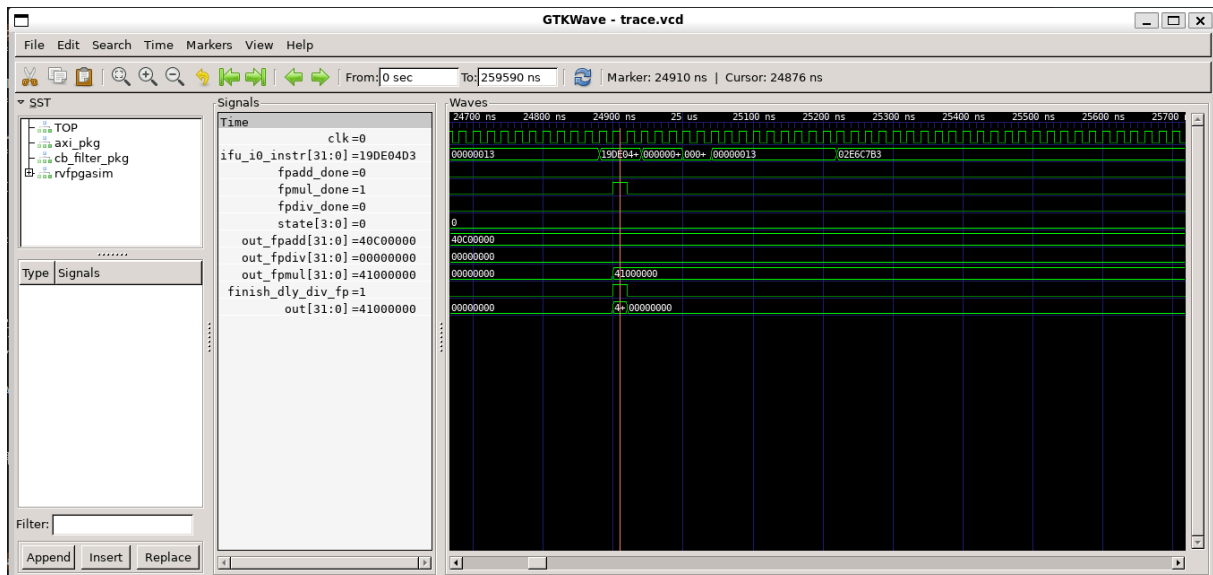
- (1) 对应的 stb*_for_fp 保持数个周期高电平；
- (2) 对应浮点模块的状态机从 get_a → get_b → ... → put_z；
- (3) 对应的 *_done 信号拉高一个周期；
- (4) out[31:0] 输出正确的结果。

5.2 实验结果

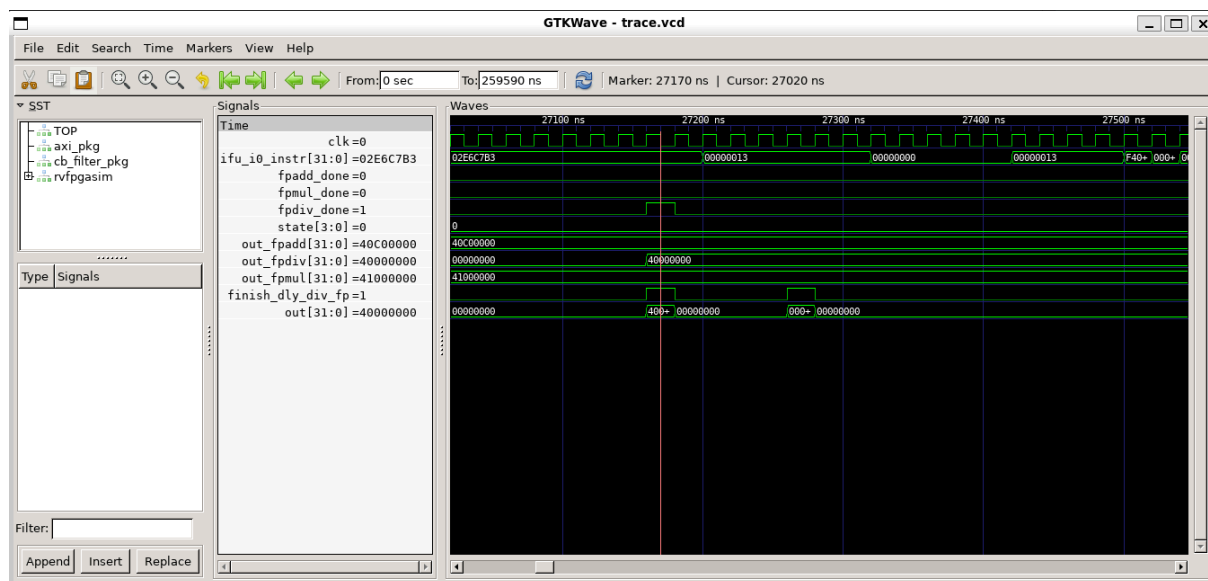
当执行浮点加法时，总输出接浮点加法器输出 0x40C00000，完成标志信号均被拉高。



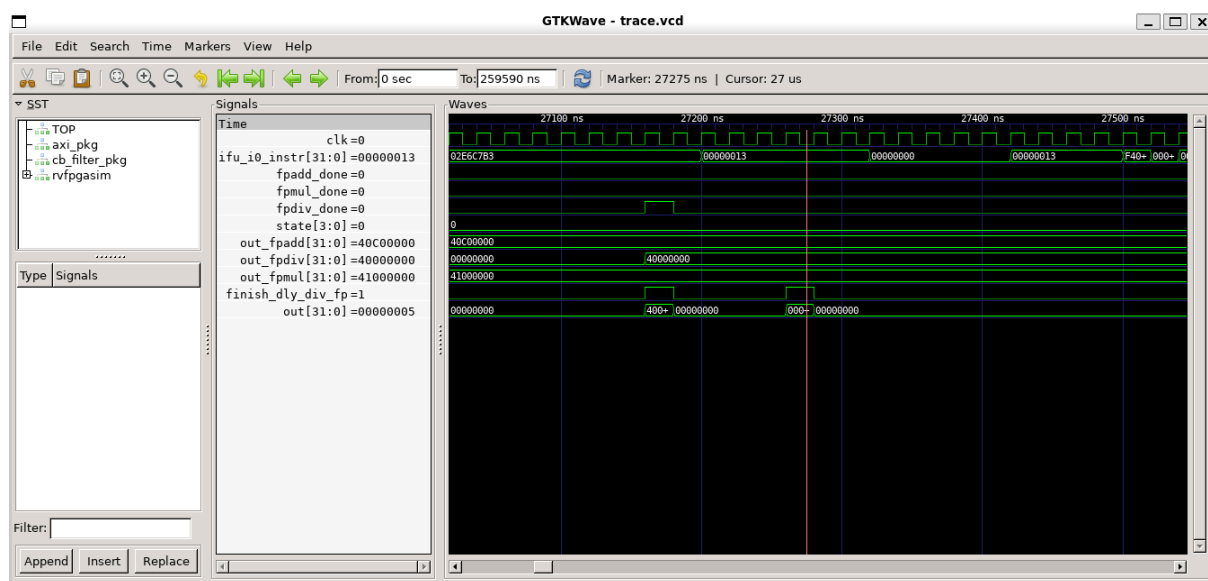
执行浮点乘法，总输出接浮点乘法器输出 0x41000000,。



执行浮点除法，总输出接浮点除法器输出 0x40000000。



最后执行原始的除法运算，连接总输出。三个浮点加法器的输出不变。在等待所有运算完成后，总输出结果如下，说明我们的浮点运算模块和测试文件设计成功。



六 问题反馈与解决

6.1 Linux 版本与兼容性问题

由于 Linux 中软件的运行需要多个依赖，如果某个软件版本过新可能导致相应依赖缺失。如果直接输入 `wsl --install` 会自动安装最新版本的 Ubuntu24.04，导致缺少测试所需软件安装运行的相关依赖，导致安装失败。经过多次测试，必须手动指定版本为 20.04 才能正常安装，这一点在第一部分中已经提及。

6.2 新增.S 测试文件问题

打开 catapult-studio 设置自己的 test.S 文件时会出现无法编译的问题。查看终端报错发现, CMake 的关联地址仍然是助教电脑上的地址, 因此需要打开 CMake.txt 文件将这些地址逐个更改为自己的项目地址 Lab18 文件夹内, 才能成功编译。